

FLASHCARDS DECK

ENRIQUECIDOS

Claude Certified Architect
Foundations Certification

Generated: 15-08-2026 18:18:14
Total Cards: 2

ÍNDICE DE PERGUNTAS

001 - Your agent needs to insert a new helper function into the middle of a ...

002 - An agent is dropped into an unfamiliar repository and asked to add a f...

PERGUNTA (ENGLISH):

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?



PERGUNTA (PORTUGUÊS):

Cenário: Seu agente precisa inserir uma nova função auxiliar no meio de um módulo utilitário de 150 linhas, entre duas funções existentes. A ferramenta Edit falha porque seu parâmetro ``old_string`` não consegue encontrar um texto único para corresponder — o arquivo possui docstrings repetitivas, nomes de variáveis e padrões estruturais. Qual é a forma mais confiável de completar essa inserção? A) Usar Edit com um ``old_string`` extremamente longo capturando 30+ linhas de contexto para garantir unicidade B) Usar o parâmetro ``replace_all`` do Edit para direcionar um padrão comum e incorporar a nova função no texto de substituição C) Usar Bash para anexar a definição da função ao final do arquivo usando sintaxe heredoc D) Usar Read para carregar o arquivo, adicionar a função no local apropriado, então Write do arquivo atualizado



ANÁLISE TÉCNICA (TECH LEAD):



EXPLICAÇÃO ACESSÍVEL (CRIANÇAS):

****Explicação:**** Imagine que você tem um grande livro e precisa adicionar uma página no meio, entre o Capítulo 2 e o Capítulo 3. Mas o livro é tão grande e tem tantos capítulos que muitos deles têm títulos parecidos. Se você tentar dizer "procure pelo padrão 'Capítulo'" para inserir a página no lugar exato, a ferramenta pode ficar confusa — porque há muitos "Capítulos" que parecem iguais. Como você adiciona a página no lugar exato sem riscos? ****Por que a alternativa D é a correta:**** A solução mais segura é pegar o livro inteiro e colocá-lo sobre a mesa. Você lê cada página, conta onde está o Capítulo 2 e o Capítulo 3, e coloca a página nova entre eles. Depois coloca o livro de volta na estante. Assim você tem ****controle total**** sobre exatamente onde a página vai, sem depender de um padrão que pode aparecer muitas vezes. Read-Modify-Write funciona assim: lê tudo, modifica no lugar certo, escreve tudo de volta. ****Por que as outras estão erradas:**** A) **■■** Usar Edit com 30+ linhas de contexto ainda depende de encontrar um padrão único — se o arquivo tem muitos trechos parecidos, você ainda fica preso ao mesmo problema. Consequência: a ferramenta ainda pode não conseguir identificar o lugar certo. B) **■■** Usar ``replace_all`` é como dizer "mude cada 'Capítulo' que você encontrar no livro inteiro". Você não muda só o lugar que queria — muda TODOS. Consequência: você bagunça o livro inteiro em vez de adicionar uma página no lugar certo. C) **■■** Usar Bash com heredoc coloca a página nova no final do livro, fora do lugar especificado. O requisito era inserir a função entre duas funções existentes, não no final. Consequência: a página fica no lugar errado e não responde ao

[illegible]

✓ A

PERGUNTA (ENGLISH):

An agent is dropped into an unfamiliar repository and asked to add a feature. The best way to orient without burning context is to:

[illegible]

PERGUNTA (PORTUGUÊS):

Cenário: Um agente é inserido em um repositório desconhecido e solicitado a adicionar um recurso. A melhor forma de se orientar sem consumir contexto é: A) Carregar todos os arquivos no contexto para que nada seja perdido. B) Ler os pontos de entrada e a estrutura do projeto, depois procurar pela área que o recurso afeta. C) Começar a editar o primeiro arquivo que pareça relacionado. D) Pedir ao usuário para explicar cada arquivo. ---

[illegible]

ANÁLISE TÉCNICA (TECH LEAD):

12345678910111213141516171819202122232425262728293031323334353637383940414243444546474849505152535455565758596061626364656667686970717273747576777879808182838485868788899091929394959697989910010110210310410510610710810911011111211311411511611711811912012112212312412512612712812913013113213313413513613713813914014114214314414514614714814915015115215315415515615715815916016116216316416516616716816917017117217317417517617717817918018118218318418518618718818919019119219319419519619719819920020120220320420520620720820921021121221321421521621721821922022122222322422522622722822923023123223323423523623723823924024124224324424524624724824925025125225325425525625725825926026126226326426526626726826927027127227327427527627727827928028128228328428528628728828929029129229329429529629729829930030130230330430530630730830931031131231331431531631731831932032132232332432532632732832933033133233333433533633733833934034134234334434534634734834935035135235335435535635735835936036136236336436536636736836937037137237337437537637737837938038138238338438538638738838939039139239339439539639739839940040140240340440540640740840941041141241341441541641741841942042142242342442542642742842943043143243343443543643743843944044144244344444544644744844945045145245345445545645745845946046146246346446546646746846947047147247347447547647747847948048148248348448548648748848949049149249349449549649749849950050150250350450550650750850951051151251351451551651751851952052152

EXPLICAÇÃO ACESSÍVEL (CRIANÇAS):

****Explicação:**** Imagine que você chega em uma cidade desconhecida e precisa entregar um pacote em uma loja específica. A melhor estratégia não é explorar cada rua e beco (isso consome muito tempo e energia!). Em vez disso, você pergunta a um morador: "Onde fica o centro? Quais são as ruas principais?" Depois, você procura a loja no mapa que montou mentalmente. Um repositório de código funciona igual — você precisa do "mapa" antes de sair correndo por todas as "ruas". ****Por que a alternativa B é a correta:**** Esta é a melhor opção porque é como ler o mapa ANTES de explorar a cidade. Os "entry points" (arquivos como `main.py`, `config/`) são como as ruas principais e os sinais de trânsito. A estrutura do projeto é o mapa. Se você estuda isso primeiro, você descobre automaticamente aonde ir. Você entende onde estão os negócios (código), como tudo se conecta, e consegue encontrar exatamente o lugar que precisa modificar sem se perder. ****Por que as outras estão erradas:**** A) **■■■** Carregar todos os arquivos queima sua energia (contexto). É como tentar memorizar cada rua, cada esquina e cada prédio da cidade de uma vez — você fica cansado e acaba não sabendo por onde começar a entregar o pacote. C) **■■■** Começar a editar sem entender o projeto é perigoso. É como entrar na primeira loja que você vê e começar a mexer nas coisas sem saber como a loja funciona — você pode quebrar algo importante ou duplicar o trabalho de alguém. D) **■■■** Pedir ao usuário para explicar cada arquivo não funciona bem porque repositórios bem organizados já contam a história por si: nomes descritivos, pastas estruturadas. Você aprende lendo sozinho — o usuário não precisa ser seu guia pessoal! ****Dica importante:**** Lembre-se: sempre que você chegar em um lugar novo (um repositório, um projeto, uma empresa), comece pelo MAPA — pergunte pela estrutura, pontos de entrada, limites principais. Este padrão funciona para tudo: aprender um novo videogame, entender como uma casa é organizada, ou navegar em um novo idioma. Estrutura primeiro, detalhes depois! ---

[illegible]

RESPOSTA CORRETA:

✓ A



FIM DO DECK

Cards neste deck: 2

Gerado em: 15-08-2026 18:18:14

Estrutura de cada card:

- PERGUNTA (ENGLISH)
- OPÇÕES (A, B, C, D)
- PERGUNTA (PORTUGUÊS)
- ANÁLISE TÉCNICA (TECH LEAD)
- EXPLICAÇÃO ACESSÍVEL (CRIANÇAS)
- RESPOSTA CORRETA

Para uso em Spaced Repetition Systems (SRS)

